DESIGN AND IMPLEMENTATION OF A

32-BIT SINGLE-CYCLE RISC-V PROCESSOR

USING VERILOG HDL

RISC-V®

SINGLE CYCLE ARCHITECTURE | 32-BIT DATAPATH | SUPPORTS RV32I INSTRUCTIONS | VERIFIED BY SIMULATION

Name : Radwa Mohamed Reda

Course : Digital Design

Course By : IEEE CUSB

IEEE

Single Cycle RV-32I Processor

Final Project

Table Of Content:

Final Project

Introduction:

The Reduced Instruction Set Computer Version Five (RISC-V) is a modern open-source Instruction Set Architecture (ISA) designed to provide a simple, modular, and extensible processor architecture. Due to its flexibility and simplicity, RISC-V has become widely used in both academic and industrial applications for processor design and computer architecture education.

This project presents the implementation of a 32-bit Single-Cycle RISC-V Processor based on the Harvard Architecture, where the instruction memory and data memory are physically separated. In a single-cycle processor, every instruction is executed completely within a single clock cycle. Therefore, all stages of instruction execution—including instruction fetch, instruction decode, register read, execution, memory access, write-back, and program counter update—are completed before the next clock edge.

The processor was designed using Verilog Hardware Description Language (HDL) following a modular approach. Each hardware component was implemented as an independent module, including the Arithmetic Logic Unit (ALU), Program Counter, Instruction Memory, Register File, Data Memory, Control Unit, Sign Extension Unit, Multiplexers, Adders, and Next Program Counter Logic. These modules were then integrated into a top-level processor module to form a complete RISC-V implementation.

The processor supports a subset of the RISC-V instruction set, including arithmetic operations, logical operations, immediate instructions, memory access instructions (Load and Store), and branch instructions. The correctness of the design was verified through simulation using a testbench that executes a Fibonacci program stored in the instruction memory.

Objective:

The objective of this project is to design, implement, and verify a 32-bit Single-Cycle RISC-V Processor using Verilog HDL according to the provided processor datapath.

The processor is required to implement all major hardware modules of the RISC-V architecture, including the Program Counter, Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Data Memory, Control Unit, Sign Extension Unit, Multiplexers, Adders, and Next Program Counter Logic. These modules are integrated into a top-level processor capable of executing instructions in a single clock cycle.

The implemented processor supports a subset of the RISC-V ISA consisting of:

- Arithmetic instructions
- Logical instructions

- Immediate instructions
- Load Word (LW)
- Store Word (SW)
- Branch Equal (BEQ)
- Branch Not Equal (BNE)
- Branch Less Than (BLT)

Finally, the processor is verified through simulation by executing a Fibonacci program loaded into the instruction memory. The simulation results are compared with the expected output to ensure that all processor modules operate correctly and interact properly within the complete datapath

Processor Specifications:

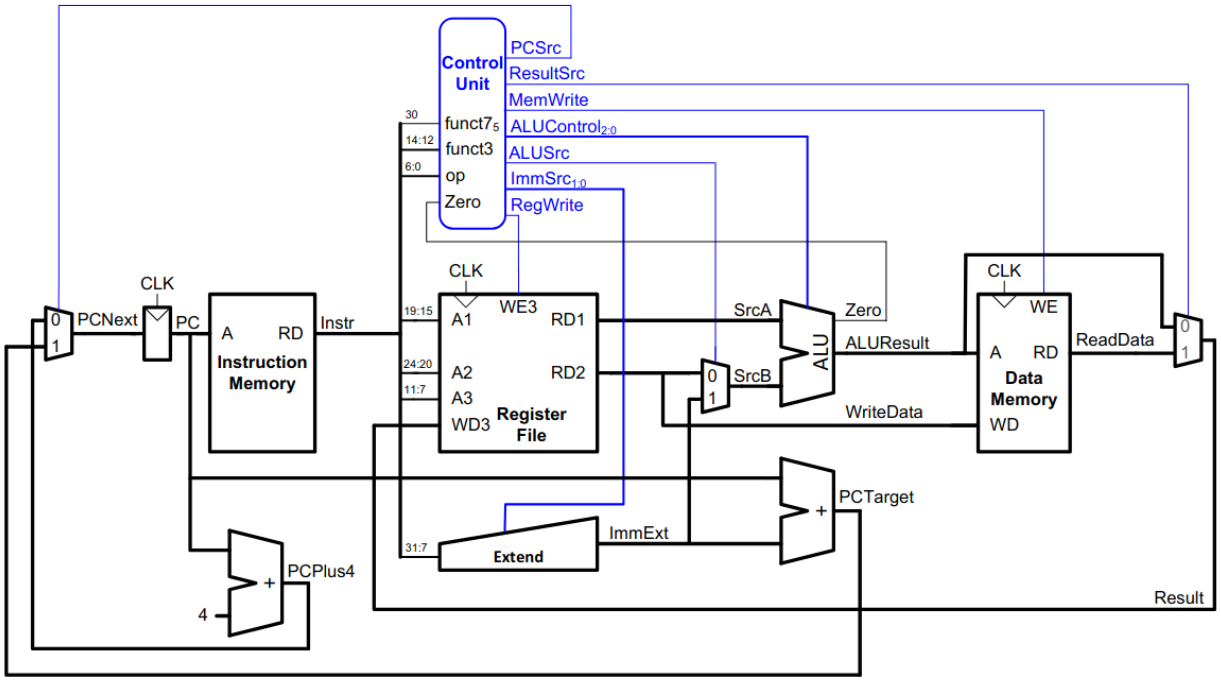
FEATURE	VALUE
ISA	RV32I (subset)
DATAPATH WIDTH	32 bits
ARCHITECTURE	Single Cycle
REGISTER FILE	32 × 32-bit
INSTRUCTION MEMORY	64 × 32-bit
DATA MEMORY	64 × 32-bit
READ TYPE	Asynchronous
WRITE TYPE	Synchronous
BRANCHES	BEQ, BNE, BLT

Final Project

Supported Instructions:

CATEGORY	INSTRUCTIONS
ARITHMETIC	ADD, SUB
LOGICAL	AND, OR, XOR
SHIFT	SLL, SRL
IMMEDIATE	ADDI, SLLI
MEMORY	LW, SW
BRANCH	BEQ, BNE, BLT

Processor Architecture:



The processor consists of the following functional units:

- Program Counter
- Instruction Memory
- Register File
- Sign Extension Unit
- ALU

- Data Memory
- Control Unit
- Next PC Logic
- Multiplexers

Modules Descriptions:

1. Adder:

```
/*
-----------------------------------------------------------
Module Name : Adder_32B

Description :
- A 32-bit combinational adder used in the RISC-V processor.
- Adds two input operands (A and B).
- Produces the arithmetic sum on the output (Sum).
- Implemented using a continuous assignment statement.
- No clock or reset is required since the operation is purely combinational.
- Parameterized by WIDTH (default = 32 bits) for flexibility.
-----------------------------------------------------------
*/

module Adder_32B #(
    parameter WIDTH = 32
) (
    input       wire    [WIDTH-1:0]     A,
    input       wire    [WIDTH-1:0]     B,
    output      wire    [WIDTH-1:0]     Sum
);

assign Sum = A + B;

endmodule
```

2. Multiplexer:

```
/*
-----------------------------------------------------------
-
Module Name : MUX_2X1

Description :
- A 2-to-1 multiplexer used in the RISC-V processor.
- Selects one of two input operands (A or B) based on the
  select signal (Sel).
- When Sel = 0, input A is forwarded to the output.
- When Sel = 1, input B is forwarded to the output.
- Implemented as a combinational logic block using an
  always @(*) statement.
- Parameterized by WIDTH (default = 32 bits) to support
  different data widths.
-----------------------------------------------------------
-
*/
```

```
module MUX_2X1 #(
    parameter WIDTH = 32
) (
    input       wire     [WIDTH-1:0]     A,
    input       wire     [WIDTH-1:0]     B,
    input       wire                     Sel,
    output      reg      [WIDTH-1:0]     Y
);

always @(*) begin
    case(Sel)
        1'b0:    Y = A;
        1'b1:    Y = B;
        default: Y = {WIDTH{1'b0}};
    endcase
end

endmodule
```

3. Program Counter:

```
/*
-----------------------------------------------------------------
-
Module Name : PC_Reg

Description :
- 32-bit Program Counter (PC) register.
- Stores the address of the current instruction.
- Loads the next program counter value (PCNext) on the
rising
  edge of the clock when the load signal is asserted.
- Supports an active-low asynchronous reset that clears the
  program counter to zero.
-----------------------------------------------------------------
-
*/

module PC_Reg #(
    parameter WIDTH = 32
) (
    input       wire       [WIDTH-1:0]       PCNext,
    input       wire                         load,
    input       wire                         clk,
    input       wire                         rst_n,
    output      reg       [WIDTH-1:0]        PC
);
```

Final Project

```
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        PC <= {WIDTH{1'b0}};
    end else if(load) begin
        PC <= PCNext;
    end
end
endmodule
```

```
/*
-----------------------------------------------------------------
-
Module Name : Next_PC_Calc

Description :
- Calculates the next value of the Program Counter (PC).
- Computes both PC + 4 and PC + ImmExt.
- Uses the PCSrc control signal to select the appropriate
  next PC value.
- PC + 4 is used for sequential execution, while PC +
ImmExt
  is used for branch instructions.
-----------------------------------------------------------------
-
*/

module Next_PC_Calc #(
    parameter WIDTH = 32
) (
    input       wire     [WIDTH-1:0]     PC,
    input       wire     [WIDTH-1:0]     ImmExt,
    input       wire                     PCSrc,
    output      wire      [WIDTH-1:0]    PCNext
);

wire     [WIDTH-1:0]     PCTarget;
wire     [WIDTH-1:0]     PCPlus4;

localparam [WIDTH-1:0] FOUR = 32'd4;

Adder_32B #(
    .WIDTH(WIDTH)
) PC_Target_Adder (
    .A(PC),
    .B(ImmExt),
    .Sum(PCTarget)
);

Adder_32B #(
    .WIDTH(WIDTH)
) PCPlus4_Adder (
    .A(PC),
    .B(FOUR),
    .Sum(PCPlus4)
);
```

```
MUX_2X1 #(
     .WIDTH(WIDTH)
) PC_Select_MUX (
     .A(PCPlus4),
     .B(PCTarget),
     .Sel(PCSrc),
     .Y(PCNext)
);
endmodule
```

4. Instruction Memory

```
/*
-------------------------------------------------------------
Module Name : Instruction_Memory

Description :
- Read-only memory (ROM) used to store program instructions.
- Provides asynchronous read access.
- Stores 64 instructions, each 32 bits wide.
- The Program Counter (PC) is connected directly to the
  address input.
- Memory is word-aligned; therefore, address bits [31:2]
  are used to index the ROM.

    Byte Address:
    0   4   8   12   16  20   24   28   32

    ROM Index:
    0   1   2    3   4   5    6   7    8
-------------------------------------------------------------
*/

module Instruction_Memory #(
    parameter WIDTH = 32,
    parameter DEPTH = 64,
    parameter ADDR_WIDTH = $clog2(DEPTH)
) (
    input       wire     [WIDTH-1:0]     A,   // 32-bit Program Counter
address
    output      wire     [WIDTH-1:0]    RD   // Instruction output
);

reg [WIDTH-1:0] ROM [0:DEPTH-1];

// Asynchronous read access
assign RD = ROM[A[ADDR_WIDTH+1:2]]; // Ignore bits [1:0] for word alignment

endmodule
```

5. Register File

```
/*
-----------------------------------------------------------
Module Name : Reg_File

Description :
- A register file used in the RISC-V processor.
- Contains 32 registers, each with 32-bit width.
- Provides two asynchronous read ports (RD1 and RD2).
- Provides one synchronous write port (WD3).
- The register data is written on the rising edge of the clock
  when the write enable signal (WE3) is high.
- Supports simultaneous reading from two registers and writing
  to one register.
- Register x0 is always read as zero according to RISC-V standard.

-----------------------------------------------------------
*/

module Reg_File #(
    parameter WIDTH = 32,
    parameter DEPTH = 32,
    parameter ADDR_WIDTH = $clog2(DEPTH)
) (
    input       wire       [ADDR_WIDTH-1:0]       A1, // Address of the first register to read
    input       wire       [ADDR_WIDTH-1:0]       A2, // Address of the second register to read
    input       wire       [ADDR_WIDTH-1:0]       A3, // Address of the register to write
    input       wire       [WIDTH-1:0]            WD3,// Data to write to the register file
    input       wire                              clk,
    input       wire                              rst_n,
    input       wire                              WE3,// Write Enable for the register file
    output      reg       [WIDTH-1:0]            RD1,// Data read from the first  registers
    output      reg       [WIDTH-1:0]            RD2 // Data read from the second registers
);

reg [WIDTH-1:0] RAM [0:DEPTH-1]; // Register file array

// write synchronously
integer i;

always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        for(i=0;i<DEPTH;i=i+1)
            RAM[i] <= 0;
    end
    else if(WE3 && (A3!=0))
        RAM[A3] <= WD3;
end

// read asynchronously
always @(*) begin
    RD1 = (A1 == 0) ? {WIDTH{1'b0}} : RAM[A1];
    RD2 = (A2 == 0) ? {WIDTH{1'b0}} : RAM[A2];
end
endmodule
```

6. Sign Extend

```
/*
-----------------------------------------------------------
Module Name : signExtend

Description :
- Extracts the immediate field from the 32-bit instruction.
- Supports different immediate formats based on the instruction type
  (I-Type, S-Type and B-Type).
- Performs sign extension to convert the extracted immediate into a
  32-bit signed value.
- The immediate extraction mode is selected using the ImmSrc control signal
  generated by the Main Decoder.
-----------------------------------------------------------
*/

module signExtend #(
    parameter INSTR_WIDTH  = 32,
    parameter IMM_WIDTH    = 32
) (
    input       wire     [INSTR_WIDTH - 1:7]     Instr,  // 32-bit instruction
    input       wire          [1:0]              ImmSrc, // Immediate source selector
    output      reg       [IMM_WIDTH - 1:0]      ImmExt  // 32-bit sign-extended immediate
);

always @(*) begin

    case(ImmSrc)
        2'b00: ImmExt = {{20{Instr[31]}}, Instr[31:20]}; // I-Type & Load Instructions
        2'b01: ImmExt = {{20{Instr[31]}}, Instr[31:25], Instr[11:7]}; // S-Type
        2'b10: ImmExt = {
                    {19{Instr[31]}},
                    Instr[31],
                    Instr[7],
                    Instr[30:25],
                    Instr[11:8],
                    1'b0
                }; // B-Type
        default: ImmExt = {IMM_WIDTH{1'b0}}; // Default case
    endcase

end

endmodule
```

7. ALU

```
module ALU_32B #(
    parameter WIDTH = 32
) (
    input       wire     [WIDTH-1:0]     SrcA,
    input       wire     [WIDTH-1:0]     SrcB,
    input       wire     [2:0]           ALUControl,

    output      wire                      Zero,
    output      wire                      SignFlag,
    output      reg      [WIDTH-1:0]     ALUResult
);

always @(*) begin

    case(ALUControl)
    3'b000:  ALUResult = SrcA  + SrcB;         // Addition
    3'b001:  ALUResult = SrcA << SrcB[4:0];   //  shift left logical
    3'b010:  ALUResult = SrcA  - SrcB;       //   Subtraction
    3'b100:  ALUResult = SrcA ^  SrcB;      //    XOR
    3'b101:  ALUResult = SrcA >> SrcB[4:0];//     shift right logical
    3'b110:  ALUResult = SrcA  | SrcB;    //      OR
    3'b111:  ALUResult = SrcA  & SrcB;   //       AND
    default: ALUResult = {WIDTH{1'b0}}; //        NOP
    endcase

end

assign Zero = (ALUResult == {WIDTH{1'b0}});
assign SignFlag = ALUResult[WIDTH-1];
endmodule
```

8. Data Memory

```
/*
-----------------------------------------------------------
Module Name : Data_Memory

Description :
- A data memory module used in the RISC-V processor.
- Implements a single read/write memory port.
- Supports asynchronous read operation and synchronous write
  operation on the rising edge of the clock.
- Data is written to address A when Write Enable (WE) is high.
- The memory stores 64 entries, each with a 32-bit word width.
- The address input A is 32-bit, matching the processor datapath.
- Word-aligned addressing is used, where address bits [7:2]
  are used to access the 64 memory locations.
- The read data output (RD) is available without waiting for
  a clock edge.
-----------------------------------------------------------
*/

module Data_Memory #(
    parameter WIDTH = 32,
    parameter DEPTH = 64
) (
    input       wire     [WIDTH-1:0]     A,
    input       wire     [WIDTH-1:0]     WD,
    input       wire                     WE,
    input       wire                     clk,
    input       wire                     rst_n,
    output      wire      [WIDTH-1:0]     RD
);

reg [WIDTH-1:0] mem [0:DEPTH-1]; // Memory array

integer i;

always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        for(i=0; i<DEPTH; i=i+1)
            mem[i] <= 0;
    end
    else if(WE)
        mem[A[7:2]] <= WD;
end

assign RD = mem[A[7:2]]; // Asynchronous read access

endmodule
```

9. Main Decoder

```
/*
-------------------------------------------------------------
Module Name : Main_Decoder

Description :
- Decodes the 7-bit instruction opcode to generate the main
  control signals for the RISC-V processor.
- Implements combinational logic using a case statement.
- Supports the following instruction types:
    * Load Word (LW)
    * Store Word (SW)
    * R-Type instructions
    * I-Type instructions
    * Branch instructions
- Generates the following control signals:
    * RegWrite
    * ImmSrc
    * ALUSrc
    * MemWrite
    * ResultSrc
    * Branch
    * ALUOp
- Unsupported opcodes generate safe default control signals.
-------------------------------------------------------------
*/

module Main_Decoder (
    input       wire     [6:0]     Opcode,

    output      reg      [1:0]     ImmSrc,
    output      reg                Branch,
    output      reg                Load,
    output      reg                ResultSrc,
    output      reg                MemWrite,
    output      reg                ALUSrc,
    output      reg                RegWrite,
    output      reg      [1:0]     ALUOp
);

  // Opcode definitions
    localparam LW     = 7'b0000011; // Load Word
    localparam SW     = 7'b0100011; // Store Word
    localparam R_TYPE = 7'b0110011; // R-Type instructions
    localparam I_TYPE = 7'b0010011; // I-Type instructions
    localparam BRANCH = 7'b1100011; // Branch instructions

    // Main Decoder Logic
    always @(*) begin
        // Default control signal values
        ImmSrc     = 2'b00;
        Branch     = 1'b0;
        Load       = 1'b1; //// Enable instruction/data access for all valid
opcodes
        ResultSrc  = 1'b0;
        MemWrite   = 1'b0;
        ALUSrc     = 1'b0;
        RegWrite   = 1'b0;
        ALUOp      = 2'b00;

        case (Opcode)
            LW: begin
                ImmSrc     = 2'b00; // I-Type immediate
                ResultSrc  = 1'b1;  // Select memory data as result
                ALUSrc     = 1'b1;  // Use immediate for ALU operation
                RegWrite   = 1'b1;  // Enable register write
                ALUOp      = 2'b00; // ALU performs addition for address
calculation
            end
```

```
            R_TYPE: begin
                RegWrite   = 1'b1;  // Enable register write
                ALUOp      = 2'b10; // ALU operation determined by funct3
and funct7 fields
            end

            I_TYPE: begin
                ImmSrc     = 2'b00; // I-Type immediate
                RegWrite   = 1'b1;  // Enable register write
                ALUSrc     = 1'b1;  // Use immediate for ALU operation
                ALUOp      = 2'b10; // ALU operation determined by funct3
field (I-Type)
            end

            BRANCH: begin
                ImmSrc     = 2'b10; // B-Type immediate
                Branch     = 1'b1;  // Enable branch operation
                ALUOp      = 2'b01; // ALU performs subtraction
            end
            default: begin
                // Default case for unsupported opcodes
                ImmSrc     = 2'b00;
                Branch     = 1'b0;
                Load       = 1'b0;  // Disable load for unsupported opcodes
                ResultSrc  = 1'b0;
                MemWrite   = 1'b0;
                ALUSrc     = 1'b0;
                RegWrite   = 1'b0;
                ALUOp      = 2'b00; // Default to addition
            end
        endcase
    end

endmodule
```

10. ALU Decoder

```
/*
-------------------------------------------------------------
Module Name : ALU_Decoder

Description :
- Generates the ALU control signal based on:
    * ALUOp from the Main Decoder.
    * funct3 field.
    * Instr[30] (funct7 bit 5).
- Implements combinational logic.
- Supports the following ALU operations:
    * ADD
    * SUB
    * Shift Left Logical (SLL)
    * Shift Right Logical (SRL)
    * XOR
    * OR
    * AND
- For memory instructions (lw, sw), the ALU performs addition.
- For branch instructions, the ALU performs subtraction for
  comparison.
- Unsupported combinations default to the ADD operation.
-------------------------------------------------------------
*/
```

Final Project

```
module ALU_Decoder (
    input       wire    [1:0]    ALUOp,
    input       wire    [2:0]    funct3,    //instr[14:12]
    input       wire             Op5,       // Opcode[5]
    input       wire             funct7_5,  //instr[30]
    output      reg     [2:0]    ALUControl
);

  // ALU operation codes
    localparam ADD = 3'b000;
    localparam SHL = 3'b001;
    localparam SUB = 3'b010;
    localparam XOR = 3'b100;
    localparam SHR = 3'b101;
    localparam OR  = 3'b110;
    localparam AND = 3'b111;

always @(*) begin
    ALUControl = ADD;

    case (ALUOp)
        2'b00: ALUControl = ADD; // For lw and sw instructions
        2'b01: ALUControl = SUB; // For branch instructions
        2'b10: begin // R-Type and I-Type instructions
            case (funct3)
                3'b000: ALUControl = (Op5 && funct7_5) ? SUB : ADD; // ADD or SUB
                3'b001: ALUControl = SHL; // Shift Left Logical
                3'b100: ALUControl = XOR; // XOR
                3'b101: ALUControl = SHR; // Shift Right Logical
                3'b110: ALUControl = OR;  // OR
                3'b111: ALUControl = AND; // AND
                default: ALUControl = ADD; // Default to ADD for unsupported funct3 values
            endcase
        end
        default: ALUControl = ADD; // Default to ADD for unsupported ALUOp values
    endcase
end

endmodule
```

11. Branch Decoder

```
/*
-------------------------------------------------------------
Branch Decision Logic

Description :
- Determines whether a branch should be taken.
- Uses the Branch control signal from the Main Decoder together
  with the ALU status flags.
- The branch operation is selected using the funct3 field.
- Supported branch instructions:
    * BEQ : Branch if Zero flag is asserted.
    * BNE : Branch if Zero flag is deasserted.
    * BLT : Branch if Sign flag is asserted.
- Implemented using a case statement controlled by funct3.
-------------------------------------------------------------
*/

module Branch_Decoder (
    input       wire     [2:0]     funct3,    //instr[14:12]
    input       wire               Zero,      // ALU Zero flag
    input       wire               Sign_Flag,      // ALU Sign flag
    input       wire               Branch,    // Branch control signal from Main Decoder
    output      reg                PCSrc      // Program Counter Source signal
);

always @(*) begin
    PCSrc = 1'b0; // Default to not taking the branch
    case (funct3)
        3'b000: PCSrc = Branch & Zero;    // BEQ: Branch if Zero flag is asserted
        3'b001: PCSrc = Branch & ~Zero; // BNE: Branch if Zero flag is deasserted
        3'b100: PCSrc = Branch & Sign_Flag; // BLT: Branch if Sign flag is asserted
        default: PCSrc = 1'b0;
    endcase
end
endmodule
```

12. Control Unit

```
module Control_Unit (
    input       wire     [6:0]          Opcode,
    input       wire     [2:0]          Funct3,
    input       wire                    Funct7_5,
    input       wire                    zero,
    input       wire                    Sign_Flag,
```

Final Project

```
    output      wire      [1:0]          ImmSrc,
    output      wire                     Branch,
    output      wire                     Load,
    output      wire                     ResultSrc,
    output      wire                     MemWrite,
    output      wire                     ALUSrc,
    output      wire                     RegWrite,
    output      wire      [2:0]          ALUControl,
    output      wire                     PCSrc
);

wire  [1:0]  ALUOp;
wire Op5;

assign Op5 = Opcode[5];

Main_Decoder main_decoder_inst (
    .Opcode(Opcode),
    .ImmSrc(ImmSrc),
    .Branch(Branch),
    .Load(Load),
    .ResultSrc(ResultSrc),
    .MemWrite(MemWrite),
    .ALUSrc(ALUSrc),
    .RegWrite(RegWrite),
    .ALUOp(ALUOp)
);

ALU_Decoder alu_decoder_inst (
    .ALUOp(ALUOp),
    .funct3(Funct3),
    .Op5(Op5),
    .funct7_5(Funct7_5),
    .ALUControl(ALUControl)
);

Branch_Decoder branch_decoder_inst (
    .funct3(Funct3),
    .Branch(Branch),
    .Zero(zero),
    .Sign_Flag(Sign_Flag),
    .PCSrc(PCSrc)
);

endmodule
```

13. RISCV_Top

```
module RISCV_Top #(
    parameter WIDTH = 32,
    parameter DEPTH = 64,
    parameter ADDR_WIDTH = $clog2(DEPTH)
) (
    input       wire     clk,
    input       wire     rst_n
);

//////////////////////////////////////////////////////
/////////////////
// internal Sign_Flagals
//////////////////////////////////////////////////////
/////////////////

wire [WIDTH-1:0] PC;
wire [WIDTH-1:0] PCNext;
wire [WIDTH-1:0] Instr;

wire [WIDTH-1:0] RD1;
wire [WIDTH-1:0] RD2;

wire [WIDTH-1:0] ImmExt;

wire [WIDTH-1:0] SrcB;

wire [WIDTH-1:0] ALUResult;

wire Zero;
wire Sign_Flag;

wire [WIDTH-1:0] ReadData;

wire [WIDTH-1:0] Result;
```

```
////////////////////////////////////////////////////////
/////////////////
// Control Sign_Flagals
////////////////////////////////////////////////////////
/////////////////

wire RegWrite;
wire ALUSrc;
wire MemWrite;
wire ResultSrc;
wire Branch;
wire PCSrc;
wire Load;

wire [1:0] ImmSrc;
wire [2:0] ALUControl;

////////////////////////////////////////////////////////
/////////////////
// Module Instantiations
////////////////////////////////////////////////////////
/////////////////

Next_PC_Calc #(
    .WIDTH(WIDTH)
) Next_PC (
    .PC(PC),
    .ImmExt(ImmExt),
    .PCSrc(PCSrc),
    .PCNext(PCNext)
);

PC_Reg #(
    .WIDTH(WIDTH)
) PC_Register (
    .PCNext(PCNext),
    .load(1'b1), // Always load the next PC value
    .clk(clk),
    .rst_n(rst_n),
    .PC(PC)
);

Instruction_Memory #(
    .WIDTH(WIDTH),
    .DEPTH(DEPTH),
    .ADDR_WIDTH(ADDR_WIDTH)
) Instruction_Mem (
    .A(PC),
    .RD(Instr)
);
```

Final Project

```
Reg_File #(
    .WIDTH(WIDTH),
    .DEPTH(32)
) Register_File (
    .clk(clk),
    .rst_n(rst_n),
    .WE3(RegWrite),
    .A1(Instr[19:15]),
    .A2(Instr[24:20]),
    .A3(Instr[11:7]),
    .WD3(Result),
    .RD1(RD1),
    .RD2(RD2)
);

MUX_2X1 #(
    .WIDTH(WIDTH)
) ALUSrc_MUX (
    .A(RD2),
    .B(ImmExt),
    .Sel(ALUSrc),
    .Y(SrcB)
);

ALU_32B #(
    .WIDTH(WIDTH)
) ALU (
    .SrcA(RD1),
    .SrcB(SrcB),
    .ALUControl(ALUControl),
    .ALUResult(ALUResult),
    .Zero(Zero),
    .SignFlag(Sign_Flag)
);

Data_Memory #(
    .WIDTH(WIDTH),
    .DEPTH(DEPTH)
) Data_Mem (
    .A(ALUResult),
    .WD(RD2),
    .WE(MemWrite),
    .clk(clk),
    .rst_n(rst_n),
    .RD(ReadData)
);
```

Final Project

```
MUX_2X1 #(
    .WIDTH(WIDTH)
) ResultSrc_MUX (
    .A(ALUResult),
    .B(ReadData),
    .Sel(ResultSrc),
    .Y(Result)
);

Control_Unit Control (
    .Opcode(Instr[6:0]),
    .Funct3(Instr[14:12]),
    .Funct7_5(Instr[30]),
    .zero(Zero),
    .Sign_Flag(Sign_Flag),
    .RegWrite(RegWrite),
    .ALUSrc(ALUSrc),
    .MemWrite(MemWrite),
    .ResultSrc(ResultSrc),
    .Branch(Branch),
    .PCSrc(PCSrc),
    .ImmSrc(ImmSrc),
    .ALUControl(ALUControl),
    .Load(Load)
);

signExtend #(
    .INSTR_WIDTH(WIDTH),
    .IMM_WIDTH(WIDTH)
) Sign_Ext (
    .Instr(Instr[31:7]),
    .ImmSrc(ImmSrc),
    .ImmExt(ImmExt)
);



endmodule
```

Final Project

Testbench:

```
`timescale 1ns/1ps

module RISCV_Top_tb;

reg clk;
reg rst_n;
integer i;

//=============================================================
// Instantiate DUT
//=============================================================
RISCV_Top DUT (
    .clk(clk),
    .rst_n(rst_n)
);

//=============================================================
// Clock Generation
//=============================================================
always #5 clk = ~clk;

//=============================================================
// Initialization
//=============================================================
initial begin
    clk   = 0;
    rst_n = 0;

    // Load Program
    $readmemh("program.txt",
DUT.Instruction_Mem.ROM);

    #20;
    rst_n = 1;
end
```

Final Project

```
//=======================================================
=
// Monitor
//=======================================================
=
initial begin
     $monitor(
     "t=%0t | PC=%h | Instr=%h | RD1=%0d | RD2=%0d |
Imm=%0d | ALUCtrl=%b | ALU=%0d | Result=%0d | Zero=%b
| Branch=%b | PCSrc=%b",
     $time,
     DUT.PC,
     DUT.Instr,
     $signed(DUT.RD1),
     $signed(DUT.RD2),
     $signed(DUT.ImmExt),
     DUT.ALUControl,
     $signed(DUT.ALUResult),
     $signed(DUT.Result),
     DUT.Zero,
     DUT.Branch,
     DUT.PCSrc
     );
end

//=======================================================
=
// Print Final Results
//=======================================================
=
initial begin

     #1000;


 $display("\n=========================================")
;
     $display("       Final Register File");

 $display("=========================================");

     for(i=0;i<8;i=i+1)
          $display("x%0d = %0d", i,
DUT.Register_File.RAM[i]);


 $display("\n=========================================")
;
     $display("       Final Data Memory");

 $display("=========================================");

     for(i=0;i<10;i=i+1)
          $display("MEM[%0d] = %0d", i,
DUT.Data_Mem.mem[i]);


 $display("\n=========================================")
;

     $stop;

end

endmodule
```

Final Project

Simulation & Results:

1. Execution Transcript

Transcript

```
# t=0 | PC=00000000 | Instr=00004033 | RD1=0 | RD2=0 | Imm=0 | ALUCtrl=100 | ALU=0 | Result=0 | Zero=1 | Branch=0 | PCSrc=0
# t=25000 | PC=00000004 | Instr=00000093 | RD1=0 | RD2=0 | Imm=0 | ALUCtrl=000 | ALU=0 | Result=0 | Zero=1 | Branch=0 | PCSrc=0
# t=35000 | PC=00000008 | Instr=00100113 | RD1=0 | RD2=0 | Imm=1 | ALUCtrl=000 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=45000 | PC=0000000c | Instr=00100193 | RD1=0 | RD2=0 | Imm=1 | ALUCtrl=000 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=55000 | PC=00000010 | Instr=00100213 | RD1=0 | RD2=0 | Imm=1 | ALUCtrl=000 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=65000 | PC=00000014 | Instr=00000293 | RD1=0 | RD2=0 | Imm=0 | ALUCtrl=000 | ALU=0 | Result=0 | Zero=1 | Branch=0 | PCSrc=0
# t=75000 | PC=00000018 | Instr=00a00313 | RD1=0 | RD2=0 | Imm=10 | ALUCtrl=000 | ALU=10 | Result=10 | Zero=0 | Branch=0 | PCSrc=0
# t=85000 | PC=0000001c | Instr=00000393 | RD1=0 | RD2=0 | Imm=0 | ALUCtrl=000 | ALU=0 | Result=0 | Zero=1 | Branch=0 | PCSrc=0
# t=95000 | PC=00000020 | Instr=00418c63 | RD1=1 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=105000 | PC=00000038 | Instr=002080b3 | RD1=0 | RD2=1 | Imm=2 | ALUCtrl=000 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=115000 | PC=0000003c | Instr=004181b3 | RD1=1 | RD2=1 | Imm=4 | ALUCtrl=000 | ALU=2 | Result=2 | Zero=0 | Branch=0 | PCSrc=0
# t=125000 | PC=00000040 | Instr=00229393 | RD1=0 | RD2=1 | Imm=2 | ALUCtrl=001 | ALU=0 | Result=0 | Zero=1 | Branch=0 | PCSrc=0
# t=135000 | PC=00000044 | Instr=0013a023 | RD1=0 | RD2=1 | Imm=0 | ALUCtrl=000 | ALU=0 | Result=0 | Zero=1 | Branch=0 | PCSrc=0
# t=145000 | PC=00000048 | Instr=00128293 | RD1=0 | RD2=1 | Imm=1 | ALUCtrl=000 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=155000 | PC=0000004c | Instr=fc62cae3 | RD1=1 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=-9 | Result=-9 | Zero=0 | Branch=1 | PCSrc=1
# t=165000 | PC=00000020 | Instr=00418c63 | RD1=2 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=1 | PCSrc=0
# t=175000 | PC=00000024 | Instr=00110133 | RD1=1 | RD2=1 | Imm=1 | ALUCtrl=000 | ALU=2 | Result=2 | Zero=0 | Branch=0 | PCSrc=0
# t=185000 | PC=00000028 | Instr=404181b3 | RD1=2 | RD2=1 | Imm=1028 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=195000 | PC=0000002c | Instr=00229393 | RD1=1 | RD2=2 | Imm=2 | ALUCtrl=001 | ALU=4 | Result=4 | Zero=0 | Branch=0 | PCSrc=0
# t=205000 | PC=00000030 | Instr=0023a023 | RD1=4 | RD2=2 | Imm=0 | ALUCtrl=000 | ALU=4 | Result=4 | Zero=0 | Branch=0 | PCSrc=0
# t=215000 | PC=00000034 | Instr=00420a63 | RD1=1 | RD2=1 | Imm=20 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=225000 | PC=00000048 | Instr=00128293 | RD1=1 | RD2=1 | Imm=1 | ALUCtrl=000 | ALU=2 | Result=2 | Zero=0 | Branch=0 | PCSrc=0
# t=235000 | PC=0000004c | Instr=fc62cae3 | RD1=2 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=-8 | Result=-8 | Zero=0 | Branch=1 | PCSrc=1
# t=245000 | PC=00000020 | Instr=00418c63 | RD1=1 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=255000 | PC=00000038 | Instr=002080b3 | RD1=1 | RD2=2 | Imm=2 | ALUCtrl=000 | ALU=3 | Result=3 | Zero=0 | Branch=0 | PCSrc=0
# t=265000 | PC=0000003c | Instr=004181b3 | RD1=1 | RD2=1 | Imm=4 | ALUCtrl=000 | ALU=2 | Result=2 | Zero=0 | Branch=0 | PCSrc=0
# t=275000 | PC=00000040 | Instr=00229393 | RD1=2 | RD2=2 | Imm=2 | ALUCtrl=001 | ALU=8 | Result=8 | Zero=0 | Branch=0 | PCSrc=0
# t=285000 | PC=00000044 | Instr=0013a023 | RD1=8 | RD2=3 | Imm=0 | ALUCtrl=000 | ALU=8 | Result=8 | Zero=0 | Branch=0 | PCSrc=0
# t=295000 | PC=00000048 | Instr=00128293 | RD1=2 | RD2=3 | Imm=1 | ALUCtrl=000 | ALU=3 | Result=3 | Zero=0 | Branch=0 | PCSrc=0
# t=305000 | PC=0000004c | Instr=fc62cae3 | RD1=3 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=-7 | Result=-7 | Zero=0 | Branch=1 | PCSrc=1
# t=315000 | PC=00000020 | Instr=00418c63 | RD1=2 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=1 | PCSrc=0
# t=325000 | PC=00000024 | Instr=00110133 | RD1=2 | RD2=3 | Imm=1 | ALUCtrl=000 | ALU=5 | Result=5 | Zero=0 | Branch=0 | PCSrc=0
# t=335000 | PC=00000028 | Instr=404181b3 | RD1=2 | RD2=1 | Imm=1028 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=345000 | PC=0000002c | Instr=00229393 | RD1=3 | RD2=5 | Imm=2 | ALUCtrl=001 | ALU=12 | Result=12 | Zero=0 | Branch=0 | PCSrc=0
# t=355000 | PC=00000030 | Instr=0023a023 | RD1=12 | RD2=5 | Imm=0 | ALUCtrl=000 | ALU=12 | Result=12 | Zero=0 | Branch=0 | PCSrc=0
# t=365000 | PC=00000034 | Instr=00420a63 | RD1=1 | RD2=1 | Imm=20 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=375000 | PC=00000048 | Instr=00128293 | RD1=3 | RD2=3 | Imm=1 | ALUCtrl=000 | ALU=4 | Result=4 | Zero=0 | Branch=0 | PCSrc=0
# t=385000 | PC=0000004c | Instr=fc62cae3 | RD1=4 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=-6 | Result=-6 | Zero=0 | Branch=1 | PCSrc=1
# t=395000 | PC=00000020 | Instr=00418c63 | RD1=1 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=405000 | PC=00000038 | Instr=002080b3 | RD1=3 | RD2=5 | Imm=2 | ALUCtrl=000 | ALU=8 | Result=8 | Zero=0 | Branch=0 | PCSrc=0
# t=415000 | PC=0000003c | Instr=004181b3 | RD1=1 | RD2=1 | Imm=4 | ALUCtrl=000 | ALU=2 | Result=2 | Zero=0 | Branch=0 | PCSrc=0
# t=425000 | PC=00000040 | Instr=00229393 | RD1=4 | RD2=5 | Imm=2 | ALUCtrl=001 | ALU=16 | Result=16 | Zero=0 | Branch=0 | PCSrc=0
# t=435000 | PC=00000044 | Instr=0013a023 | RD1=16 | RD2=8 | Imm=0 | ALUCtrl=000 | ALU=16 | Result=16 | Zero=0 | Branch=0 | PCSrc=0
# t=445000 | PC=00000048 | Instr=00128293 | RD1=4 | RD2=8 | Imm=1 | ALUCtrl=000 | ALU=5 | Result=5 | Zero=0 | Branch=0 | PCSrc=0
# t=455000 | PC=0000004c | Instr=fc62cae3 | RD1=5 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=-5 | Result=-5 | Zero=0 | Branch=1 | PCSrc=1
# t=465000 | PC=00000020 | Instr=00418c63 | RD1=2 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=1 | PCSrc=0
# t=475000 | PC=00000024 | Instr=00110133 | RD1=5 | RD2=8 | Imm=1 | ALUCtrl=000 | ALU=13 | Result=13 | Zero=0 | Branch=0 | PCSrc=0
# t=485000 | PC=00000028 | Instr=404181b3 | RD1=2 | RD2=1 | Imm=1028 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=495000 | PC=0000002c | Instr=00229393 | RD1=5 | RD2=13 | Imm=2 | ALUCtrl=001 | ALU=20 | Result=20 | Zero=0 | Branch=0 | PCSrc=0
# t=505000 | PC=00000030 | Instr=0023a023 | RD1=20 | RD2=13 | Imm=0 | ALUCtrl=000 | ALU=20 | Result=20 | Zero=0 | Branch=0 | PCSrc=0
# t=515000 | PC=00000034 | Instr=00420a63 | RD1=1 | RD2=1 | Imm=20 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=525000 | PC=00000048 | Instr=00128293 | RD1=5 | RD2=8 | Imm=1 | ALUCtrl=000 | ALU=6 | Result=6 | Zero=0 | Branch=0 | PCSrc=0
# t=535000 | PC=0000004c | Instr=fc62cae3 | RD1=6 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=-4 | Result=-4 | Zero=0 | Branch=1 | PCSrc=1
# t=545000 | PC=00000020 | Instr=00418c63 | RD1=1 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=555000 | PC=00000038 | Instr=002080b3 | RD1=8 | RD2=13 | Imm=2 | ALUCtrl=000 | ALU=21 | Result=21 | Zero=0 | Branch=0 | PCSrc=0
# t=565000 | PC=0000003c | Instr=004181b3 | RD1=1 | RD2=1 | Imm=4 | ALUCtrl=000 | ALU=2 | Result=2 | Zero=0 | Branch=0 | PCSrc=0
# t=575000 | PC=00000040 | Instr=00229393 | RD1=6 | RD2=13 | Imm=2 | ALUCtrl=001 | ALU=24 | Result=24 | Zero=0 | Branch=0 | PCSrc=0
# t=585000 | PC=00000044 | Instr=0013a023 | RD1=24 | RD2=21 | Imm=0 | ALUCtrl=000 | ALU=24 | Result=24 | Zero=0 | Branch=0 | PCSrc=0
# t=595000 | PC=00000048 | Instr=00128293 | RD1=6 | RD2=21 | Imm=1 | ALUCtrl=000 | ALU=7 | Result=7 | Zero=0 | Branch=0 | PCSrc=0
# t=605000 | PC=0000004c | Instr=fc62cae3 | RD1=7 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=-3 | Result=-3 | Zero=0 | Branch=1 | PCSrc=1
# t=615000 | PC=00000020 | Instr=00418c63 | RD1=2 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=1 | PCSrc=0
# t=625000 | PC=00000024 | Instr=00110133 | RD1=13 | RD2=21 | Imm=1 | ALUCtrl=000 | ALU=34 | Result=34 | Zero=0 | Branch=0 | PCSrc=0
# t=635000 | PC=00000028 | Instr=404181b3 | RD1=2 | RD2=1 | Imm=1028 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=645000 | PC=0000002c | Instr=00229393 | RD1=7 | RD2=34 | Imm=2 | ALUCtrl=001 | ALU=28 | Result=28 | Zero=0 | Branch=0 | PCSrc=0
# t=655000 | PC=00000030 | Instr=0023a023 | RD1=28 | RD2=34 | Imm=0 | ALUCtrl=000 | ALU=28 | Result=28 | Zero=0 | Branch=0 | PCSrc=0
# t=665000 | PC=00000034 | Instr=00420a63 | RD1=1 | RD2=1 | Imm=20 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=675000 | PC=00000048 | Instr=00128293 | RD1=7 | RD2=21 | Imm=1 | ALUCtrl=000 | ALU=8 | Result=8 | Zero=0 | Branch=0 | PCSrc=0
```

Final Project

```
# t=675000 | PC=00000048 | Instr=00128293 | RD1=7 | RD2=21 | Imm=1 | ALUCtrl=000 | ALU=8 | Result=8 | Zero=0 | Branch=0 | PCSrc=0
# t=685000 | PC=0000004c | Instr=fc62cae3 | RD1=8 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=-2 | Result=-2 | Zero=0 | Branch=1 | PCSrc=1
# t=695000 | PC=00000020 | Instr=00418c63 | RD1=1 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=705000 | PC=00000038 | Instr=002080b3 | RD1=21 | RD2=34 | Imm=2 | ALUCtrl=000 | ALU=55 | Result=55 | Zero=0 | Branch=0 | PCSrc=0
# t=715000 | PC=0000003c | Instr=004181b3 | RD1=1 | RD2=1 | Imm=4 | ALUCtrl=000 | ALU=2 | Result=2 | Zero=0 | Branch=0 | PCSrc=0
# t=725000 | PC=00000040 | Instr=00229393 | RD1=8 | RD2=34 | Imm=2 | ALUCtrl=001 | ALU=32 | Result=32 | Zero=0 | Branch=0 | PCSrc=0
# t=735000 | PC=00000044 | Instr=0013a023 | RD1=32 | RD2=55 | Imm=0 | ALUCtrl=000 | ALU=32 | Result=32 | Zero=0 | Branch=0 | PCSrc=0
# t=745000 | PC=00000048 | Instr=00128293 | RD1=8 | RD2=55 | Imm=1 | ALUCtrl=000 | ALU=9 | Result=9 | Zero=0 | Branch=0 | PCSrc=0
# t=755000 | PC=0000004c | Instr=fc62cae3 | RD1=9 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=-1 | Result=-1 | Zero=0 | Branch=1 | PCSrc=1
# t=765000 | PC=00000020 | Instr=00418c63 | RD1=2 | RD2=1 | Imm=24 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=1 | PCSrc=0
# t=775000 | PC=00000024 | Instr=00110133 | RD1=34 | RD2=55 | Imm=1 | ALUCtrl=000 | ALU=89 | Result=89 | Zero=0 | Branch=0 | PCSrc=0
# t=785000 | PC=00000028 | Instr=404181b3 | RD1=2 | RD2=1 | Imm=1028 | ALUCtrl=010 | ALU=1 | Result=1 | Zero=0 | Branch=0 | PCSrc=0
# t=795000 | PC=0000002c | Instr=00229393 | RD1=9 | RD2=89 | Imm=2 | ALUCtrl=001 | ALU=36 | Result=36 | Zero=0 | Branch=0 | PCSrc=0
```

```
# t=805000 | PC=00000030 | Instr=0023a023 | RD1=36 | RD2=89 | Imm=0 | ALUCtrl=000 | ALU=36 | Result=36 | Zero=0 | Branch=0 | PCSrc=0
# t=815000 | PC=00000034 | Instr=00420a63 | RD1=1 | RD2=1 | Imm=20 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=1
# t=825000 | PC=00000048 | Instr=00128293 | RD1=9 | RD2=55 | Imm=1 | ALUCtrl=000 | ALU=10 | Result=10 | Zero=0 | Branch=0 | PCSrc=0
# t=835000 | PC=0000004c | Instr=fc62cae3 | RD1=10 | RD2=10 | Imm=-44 | ALUCtrl=010 | ALU=0 | Result=0 | Zero=1 | Branch=1 | PCSrc=0
# t=845000 | PC=00000050 | Instr=00000000 | RD1=0 | RD2=0 | Imm=0 | ALUCtrl=000 | ALU=0 | Result=0 | Zero=1 | Branch=0 | PCSrc=0
```

During each clock cycle, the processor performs the following operations:

1. The Program Counter (PC) points to the address of the current instruction in the Instruction Memory.
2. The instruction is fetched from memory using the current PC value.
3. The fetched instruction is decoded, and the source register addresses, destination register address, and immediate value are extracted.
4. The Control Unit generates the required control signals according to the instruction type.
5. The Register File reads the values of the source registers (RD1 and RD2).
6. The Immediate Generator produces the sign-extended immediate value when required.
7. The ALU performs the selected arithmetic or logical operation, and the result is generated.
8. For memory instructions, the ALU result is used as the memory address for reading or writing data.
9. The result is written back to the destination register if the instruction requires a write-back operation.
10. Finally, the next value of the Program Counter is determined:

- PC = PC + 4 for normal sequential execution.
- PC = PC + Immediate when a branch instruction is taken (Branch = 1 and PCSrc = 1).

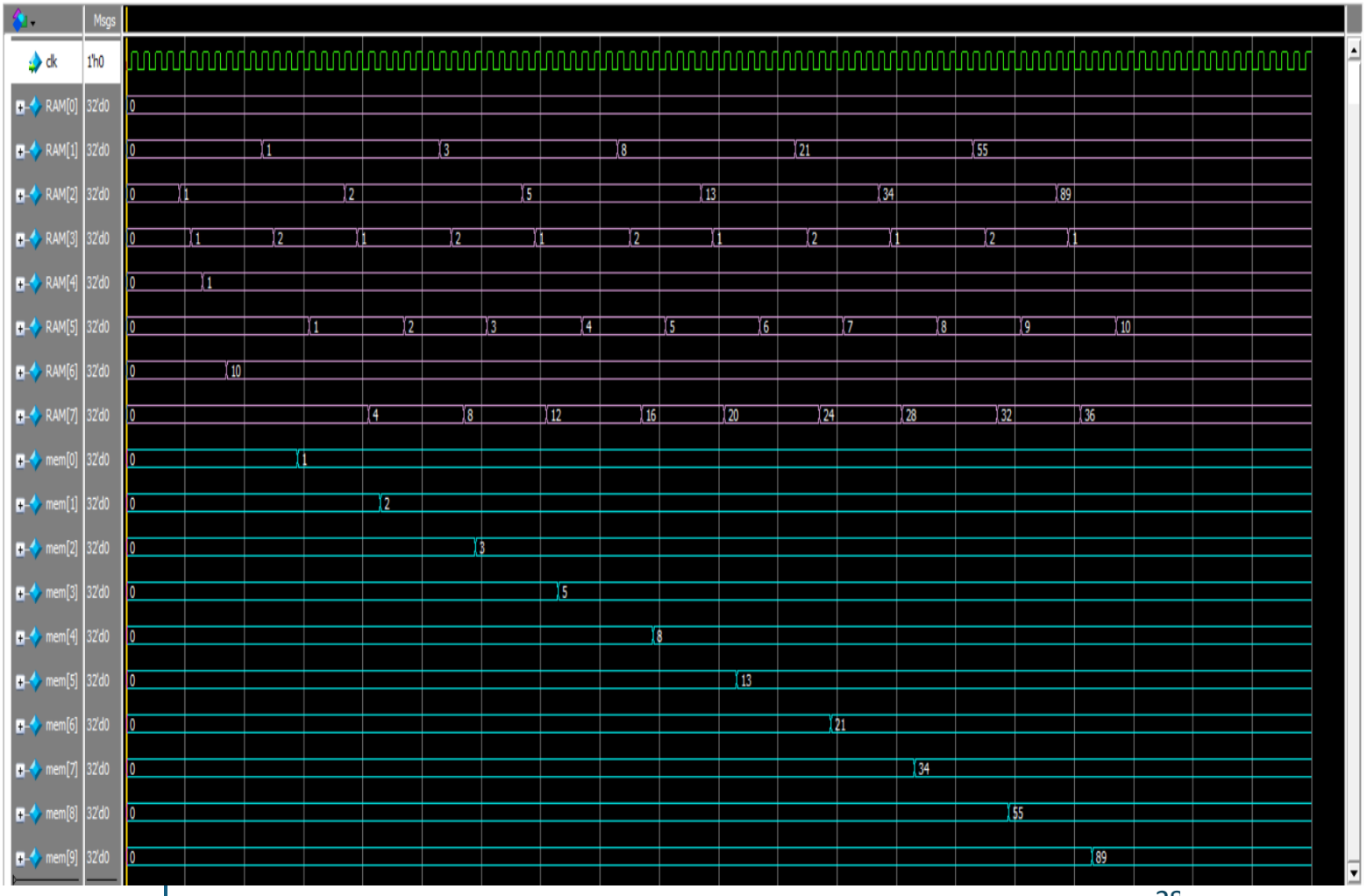
Final Project

2. Final Register File and Data Memory

After program execution, the register file contains the expected final values. The data memory stores the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21, 34, 55, 89), confirming the correct execution of arithmetic, memory, and control instructions.

```
Transcript
# ========================================
#        Final Register File
# ========================================
# x0 = 0
# x1 = 55
# x2 = 89
# x3 = 1
# x4 = 1
# x5 = 10
# x6 = 10
# x7 = 36
#
# ========================================
#        Final Data Memory
# ========================================
# MEM[0] = 1
# MEM[1] = 2
# MEM[2] = 3
# MEM[3] = 5
# MEM[4] = 8
# MEM[5] = 13
# MEM[6] = 21
# MEM[7] = 34
# MEM[8] = 55
# MEM[9] = 89
#
# ========================================
```

3. Waveform Verification



The waveform verifies the processor timing during execution. The clock controls synchronous operations, while memory locations are updated sequentially after each Store Word instruction. The waveform confirms the correct timing and functionality of the processor datapath.

Conclusion

The simulation results verify the correct functionality of the proposed single-cycle RISC-V processor. All supported instructions were executed successfully, and the execution transcript, waveform, register file, and data memory contents matched the expected behavior, confirming the correctness of the processor design.